Real-world (rebuild Doc04's extraction, LCEL-style) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The mapping, and the two versions
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The mapping, and the two versions

Doc04's raw call sends **two** messages: a system message (`EXTRACT_PROMPT`) and the user's text. `from_template(...)` builds only one message, so use `ChatPromptTemplate.from_messages([("system", EXTRACT_PROMPT), ("human", "{text}")])` — import `EXTRACT_PROMPT` from `chat_client.py` so both versions get the same words.

What does "match" mean for two model calls? A **name** has one right answer, so both versions must agree on it. **Intent** is free wording ("reset her password" vs. "reset password") that can differ between any two calls — even two raw ones. Compare `raw_result.name == lcel_result.name`.

Then split the work: `run_comparison(test_inputs) -> list[ComparisonRow]` computes, `main()` reports. That's the shape the Build Task copies.

Hint 2 — The plan, and almost the whole thing

```
@dataclass
class ComparisonRow: input_text, raw_result, lcel_result, matched

function run_comparison(test_inputs) -> list[ComparisonRow]:
    client, lcel_chain
    for each text:
        run both, matched = same name
        add a ComparisonRow
    return the rows
```

```
main: run_comparison, count matched, print any mismatch
```

Write it, run it, then compare against the Solution.

Full solution: Show me the solution